Evaluating the Impact of Discovered Causal Structures on Structure-Aware Shapley Methods: An Experimental Pipeline

Juan David Rios Garcia 2026

Abstract. Structure-aware Shapley methods promise more faithful feature attributions by embedding causal graph topology into the credit-assignment process. In practice the causal graph is rarely known: practitioners must infer it from data using automated discovery algorithms whose outputs are noisy and error-prone. This paper introduces an experimental pipeline that feeds automatically discovered graphs (PC and DirectLiNGAM) into three structure-aware methods, Asymmetric Shapley (ASV), Causal Shapley (CSV), and Shapley Flow, and measures attribution quality degradation along two dimensions: Magnitude Divergence (ΔM) and Sign Disagreement (D), each against three reference configurations. Experiments on a 50-feature confounded synthetic dataset and the Sachs cell-signaling benchmark show that ASV is the most stable under graph errors(both ΔM_{oracle} and D_{oracle} are the lowest divergence in all comparisons), while CSV and Flow amplify discovery errors into large attribution shifts. The pipeline also provides a dual-signal diagnostic, based on joint ΔM_{base} and ΔM_{disc}, standardized metrics, that identifies structurally unreliable nodes without needing a ground-truth graph, and adapt to any dataset and model output dimensions.

1 Introduction

Explainable AI has moved from a research preference to a deployment requirement across critical decisioning domains such as credit scoring, healthcare, and public resource allocation, where model decisions must be justified to affected individuals and regulatory bodies [1]. Shapley values have emerged as a gold standard for local feature attribution, but traditional implementations assume feature independence, which in practice means treating all permutations as equally probable during coalition formation, or just sampling marginal distributions when they are correlated, this could give a false credit when the true generative process involves causal dependencies among inputs [4, 5].

Recent literature has introduced structure-aware Shapley frameworks that embed causal graph topology into the attribution process or modify the sampling strategy to capture dependencies. Asymmetric Shapley Values (ASV) restrict permutations to causal orderings [2]; Causal Shapley Values (CSV) replace observational marginalization with Pearl's do-calculus [4]; and Shapley Flow reframes credit assignment from nodes to directed edges [11]. All three promise attributions more faithful to the underlying causal mechanism, but every one of these methods treats the causal graph as given.

In practice the graph is almost never directly observable: practitioners must rely on automated discovery algorithms such as the PC algorithm [10] and DirectLiNGAM [9] to infer it from observational data. These estimated graphs are inherently noisy, subject to false-positive and false-negative edge errors, and sensitive to assumption violations [3].

We build an experimental pipeline that feeds automatically discovered graphs into the three methods and measures, for the first time, how much attribution quality degrades when the graph is imperfect. Evaluation is performed on a controlled synthetic dataset (50 features, known linear causal structure) and the real-world Sachs cell-signaling dataset [7], where an established consensus DAG allows external validation.

To answer the central question consistently we track two dimensions: the *magnitude* of the attribution assigned to a feature and the *sign* (direction) of that attribution. A graph that merely rescales attributions preserves the qualitative story told to an affected individual; a graph that inverts signs tells the opposite story. Each dimension is operationalized by a single metric used identically across three comparison configurations (vs. baseline, vs. oracle, and cross-discovery), establishing a universal diagnostic framework.

2 Conceptual Framework

2.1 Explaining Models, Not Nature

One distinction matters before going any further, the goal here is not to fully recover the physical laws of the system the model was trained on, but to capture enough of its causal structure to make the attribution computation more honest [1]. A practitioner does not need a complete causal model of the world; they need a plausible graph of how the input features relate to each other so that structure-aware methods can encode those relationships as constraints, rather than treating every feature as an independent, interchangeable partner. Physical interventions are not required and often not possible; what matters is that the graph reflects the real dependencies well enough to guide the credit-assignment process in the right direction.

2.2 Structural Causal Models and Credit Attribution

A Structural Causal Model (SCM) [6] describes a system as a set of equations $x_i = f_i(\mathrm{Pa}(x_i), \varepsilon_i)$ where each variable is determined by its causal parents and independent noise. When a full SCM is available, credit attribution becomes more precise: you can intervene on a variable, fixing it to a specific value while cutting its incoming edges, and trace how that change propagates through the graph to the final prediction. This is the intuition behind CSV and Shapley Flow, both of which are designed around this causal intervention idea. CSV approximates the post-interventional distribution using Pearl's do-operator for each coalition evaluation; Flow routes credit along the actual graph edges so that structural paths determine how much each node contributes. In practice, a fully specified SCM is almost never available. The implementations here do not reconstruct structural equations: CSV uses a conditional Gaussian approximation along discovered parent sets, and Flow uses a binary foreground/background activation rule. Both methods still benefit from the graph topology, but the credit assignment is an approximation of the full interventional ideal rather than an exact application of it.

2.3 Causal Discovery

Two prominent discovery algorithms accessible to any practitioner are used in this pipeline.Both algorithms operate on the X-only training partition.

PC [10]. A constraint-based method that recovers the causal skeleton through conditional independence tests (Fisher-z, $\alpha = 0.05$) and applies Meek's orientation rules. Undirected edges are resolved by orienting them from lower to higher feature index.

DirectLiNGAM [9]. A functional causal model that exploits linear non-Gaussian structures to simultaneously identify causal ordering and structural coefficients. Edges with $|\text{coefficient}| < 0.10$ are pruned.

2.4 Traditional Shapley Values

Shapley values originate in cooperative game theory as the unique fair allocation of total payoff among players [8]. The marginal contribution of feature i to coalition S is $v(S \cup \{i\}) - v(S)$, where $v(S) = \mathbb{E}[f(X) \mid X_S = x_S]$. Averaging over all orderings gives:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\,(|N| - |S| - 1)!}{|N|!} \Big[v(S \cup \{i\}) - v(S) \Big]. \tag{1}$$

The formula is the unique solution satisfying four axioms: *Efficiency* (attributions sum to the total model output minus the baseline); *Null Player* (a feature that never changes the coalition value receives zero credit); *Linearity* (attributions for a weighted sum of games equal the weighted sum of their attributions); and *Symmetry* (two features with identical marginal contributions to every coalition receive equal attribution). Together these axioms guarantee a principled, unambiguous allocation: no matter how complex the model, the credit is divided exhaustively, no player is penalized for doing nothing, and identical contributions earn identical rewards. Symmetry is especially important for its fairness interpretation, it prevents arbitrary tie-breaking and ensures that attribution differences reflect genuine differences in predictive contribution rather than the order in which features were considered.

Where Symmetry becomes problematic is when the input features are not, in fact, interchangeable. If X_i is a causal parent of X_j, treating them as symmetric partners means the uniform average over orderings includes orderings where X_j appears before X_i, orderings that reverse the real generative sequence. These orderings let X_j absorb credit that, causally, belongs to X_i, because in the true data-generating process X_j's values are partly determined by X_i. The result is a systematic misattribution from upstream causes to downstream effects whenever features are correlated through causal structure.

2.5 Structure-Aware Methods

All three methods preserve Efficiency, Linearity, and Null Player while modifying Symmetry to accommodate causal topologies.

Asymmetric Shapley (ASV). Replaces the uniform distribution over orderings with a distribution placing mass only on topological orderings of the feature DAG [2]:

$$\phi_i(\text{ASV}) = \mathbb{E}_{\pi \sim U(\Pi_{\text{topo}})}\left[v(P_i^\pi \cup \{i\}) - v(P_i^\pi)\right]. \quad (2)$$

The coalition value remains observational; the graph enters only through the permutation set Π_{topo}.

Causal Shapley (CSV). Keeps the symmetric formulation but replaces the observational value function with an interventional one based on Pearl's do-operator [4]:

$$v_{\text{do}}(S) = \mathbb{E}[f(X) \mid \text{do}(X_S = x_S)]. \quad (3)$$

CSV uses v_{do} in the standard Shapley weighting formula. The interventional value is estimated by post-interventional sampling along the graph-defined parent sets using a conditional Gaussian approximation (Appendix B).

Shapley Flow. Moves from node-based to edge-based attribution [11]. The players are the directed graph edges:

$$\phi_{u\to v} = \mathbb{E}_{\pi \sim U(\Pi_E)}\left[V(E^\pi_{u\to v} \cup \{u \to v\}) - V(E^\pi_{u\to v})\right], \quad (4)$$

and node importance is $\phi_u = \sum_{v:\, u\to v \in E} \phi_{u\to v}$. Each node takes its foreground value if it has any active incident edge; otherwise its background value. With no graph there is no game to play.

3 Experimental Setup

Figure 1 shows the end-to-end experimental pipeline. Two datasets are evaluated across three structure-aware Shapley methods under three causal graph sources; the resulting attributions are measured along magnitude and sign dimensions.

3.1 Datasets

Synthetic (Linear-Confounded). Strictly linear structural equations $X_j = \sum_{i\in \text{Pa}(j)} w_{ij} X_i + \varepsilon_j$ ($\varepsilon_j \sim \mathcal{N}(0, 0.5)$, coefficients $\in \text{Uniform}(0.5, 2.0)$). Erdős–Rényi graph ($p = 0.07$, 87 $X \to X$ edges, 50 features, 15 direct parents of Y). Five hidden confounders each influence 2–3 features, creating a noisy scenario for both discovery algorithms. 1000 observations.

Sachs Cell Signaling [7]. 7,466 simultaneous measurements of 11 protein concentrations in stimulated human T-cells. Akt (`akt`) is the regression target; 10 proteins are input features. A consensus reference DAG (17 $X \to X$ edges, 3 direct $X \to Y$ edges) provides oracle ground truth. Preprocessing: log1p transformation, then a Standard Scaler fit on the training partition only. Both datasets use an 80/20 split, seed 42.

3.2 Predictive Model

LightGBM regressor fitted with Optuna hyperparameter optimization (50 TPE trials, RMSE objective). Feature selection is disabled so all features participate in the Shapley computation. Table 1 reports performance; $\hat{\sigma}$ is the standard deviation of model predictions over the test set and normalizes all magnitude metrics; the normalization rationale is explained

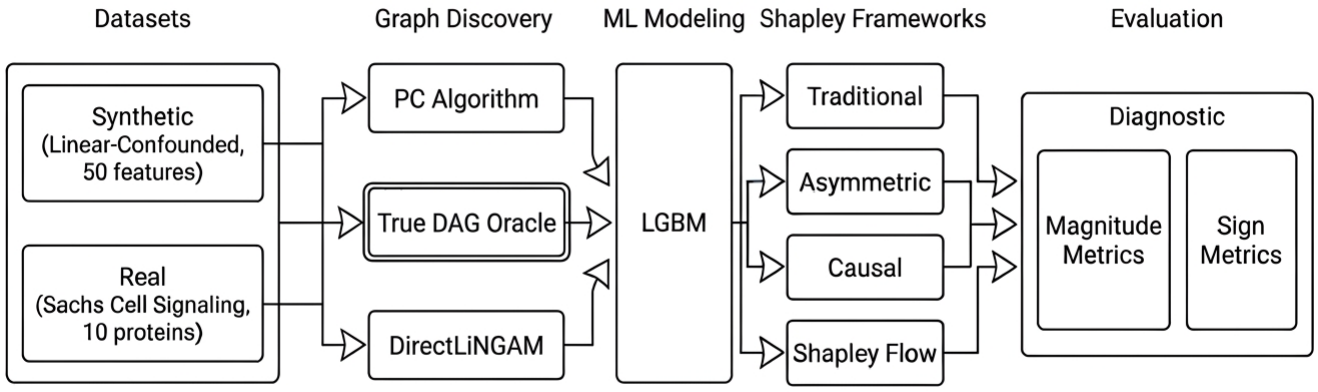


Figure 1: Experimental pipeline overview. End-to-end flow from data generation and causal discovery through Shapley computation to evaluation, showing how the two datasets, three graph sources, and three structure-aware methods feed into the magnitude and sign metrics.

Table 1: Model performance. $\hat{\sigma}$ normalizes all ΔM metrics.

Dataset	R^2	RMSE	MAE	$\hat{\sigma}$
Synthetic	0.900	1.72	1.36	4.71
Sachs	0.679	88.78	17.82	116.76

in Section 3.4.

Table 2: What each method changes relative to Traditional Shapley.

Prop.	Trad.	ASV	CSV	Flow
Players	Features	Features	Features	Edges
Perms.	$N!$ ord.	Restricted	Restricted	$\lvert E\rvert$+1
Value fn.	Obs.	Obs.	Intervent.	Fg/bg act.
Graph role	None	Ordering	Order+cond.	Players

3.3 Discovery and Computation Settings

Both discovery algorithms are configured as described in Section 2. Because neither algorithm produces $X \to Y$ edges, an augmentation step appends direct $X_i \to Y$ edges for every feature, uniformly applied to the True DAG as well, guaranteeing Shapley Flow has a route to accumulate credit for every node.

All Shapley methods use Monte Carlo approximation with $T = 100$ permutations. Background reference: 30% of training rows (240 synthetic; 1,791 Sachs). Up to 100 test instances explained. CSV uses $M = 10$ inner interventional samples per coalition. Table 2 summarizes what each method changes relative to Traditional Shapley.

3.4 Evaluation Metrics

Every comparison asks two questions about a (method, graph) result against a reference: did the attribution change in **magnitude**, and did it change in **direction**? Let $\phi_{i,f}^{m,G}$ be the Shapley value for feature f, instance i, method m, graph G.

Magnitude Divergence (ΔM). At instance level, the signed gap between attribution magnitudes is $\delta_{i,f}^{\text{ref}} = |\phi_{i,f}^{m,G}| - |\phi_{i,f}^{\text{ref}}|$. Squaring and averaging over all explained instances $\mathcal{I}$ gives a feature-level root-mean-square divergence, then divided by $\hat{\sigma}$:

$$\Delta M_f^{\text{ref}} = \frac{1}{\hat{\sigma}} \sqrt{\frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \left(\delta_{i,f}^{\text{ref}}\right)^2}. \tag{5}$$

The global dataset-level score averages across all N features: $\Delta M^{\text{ref}} = \frac{1}{N} \sum_f \Delta M_f^{\text{ref}}$.

The denominator $\hat{\sigma}$ is the standard deviation of model predictions over the test set (reported in Table 1). Dividing by $\hat{\sigma}$ expresses every divergence as a fraction of the model's own output spread, making the

numbers directly comparable across datasets, models, and output units. A value of $\Delta M = 0.10$ means the typical attribution shift for that feature equals 10% of the model's prediction spread, regardless of whether the model outputs loan amounts in thousands of dollars or protein concentrations in arbitrary units. Without this normalization, the synthetic and Sachs numbers ($\hat{\sigma} = 4.71$ vs. 116.76) would be on completely different scales and no comparison would be meaningful.

The feature-level view (ΔM_f^{ref}) is what drives the case studies in Section 4.5: it isolates which specific nodes are most destabilized by graph errors. The dataset-level view (ΔM^{ref}) is what drives the scatter-plot comparisons in Sections 4.2–4.4.

Sign Disagreement (D). The fraction of (instance, feature) pairs where the attribution flips sign relative to the reference. $D \in [0, 1]$; $D = 0$ means every attribution points in the same direction as the reference; $D = 0.33$ means one in three attributions is inverted. Sign flips are qualitatively more damaging than magnitude shifts: a rescaled attribution still tells the same directional story to an affected individual; an inverted one tells the opposite story.

Three reference configurations. The same two metrics are computed against three different references, each answering a distinct practical question:

- **vs. Traditional (ΔM_{base}, D_{base}):** How much does injecting *any* causal graph change attributions relative to the graph-free baseline? A large value here means the method is structurally sensitive to the graph: it is doing a lot of additional work beyond standard Shapley. Results in Section 4.2.
- **vs. Oracle (ΔM_{oracle}, D_{oracle}):** How far does a discovered graph push attributions from what the True (or consensus) DAG would have produced? This measures the cost of graph imperfection directly. Requires a ground-truth graph, so it is only available in controlled settings. Results in Section 4.3.
- **PC vs. LiNGAM (ΔM_{disc}, D_{disc}):** How much do attributions change when the same method is fed two different discovered graphs from the same data? No ground truth is needed—this is a purely observational stability check available to any practitioner. Results in Section 4.4.

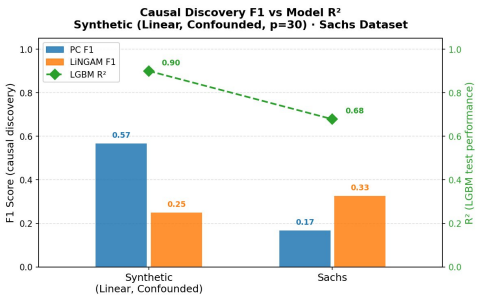


Figure 2: Causal discovery F1 ($X \to X$ edges) versus LightGBM test R^2, synthetic vs. Sachs. Bars give PC and LiNGAM F1 on each dataset; the dashed line tracks model R^2. The discovery-quality ordering of the two algorithms reverses between tracks even as model fit declines from synthetic to real data.

Only the *base* and *disc* metrics are available in a real deployment (oracle requires a known ground-truth graph). The oracle metrics are included here because the controlled experimental setting allows validating the practical diagnostics against a known reference.

4 Results and Empirical Analysis

4.1 Causal Discovery Performance

On the synthetic track, PC outperforms LiNGAM (F1: 0.567 vs. 0.250): conditional independence tests partially block confounder-induced paths, while LiNGAM's functional model conflates them with direct paths, adding 81 false-positive edges. On Sachs the ranking reverses (F1: LiNGAM 0.326 vs. PC 0.167): log-transformed proteins retain non-Gaussian marginals that LiNGAM exploits, while PC's Fisher-z test loses discriminative power when the massive sample size makes it hypersensitive to distributional violations [3]. Both algorithms perform substantially worse on the real data despite the larger sample size.

4.2 Graph-Free Baseline Deviation

This evaluation block compares each structure-aware method and discovered-graph combination against the Traditional Shapley baseline (ΔM_{base}, D_{base}) to verify the expected theoretical behaviour of each

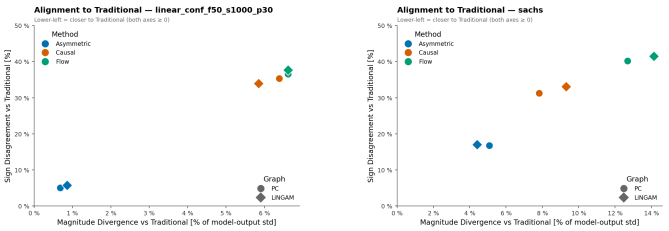
method and validate that this behaviour holds on real data. The theoretical sensitivity ordering is driven by how deeply each method couples to the graph: ASV uses discovered edges only to restrict the permutation space while keeping the observational value function unchanged, so its baseline deviation depends entirely on how many orderings the graph forbids; CSV is more sensitive because the graph enters the value function itself, driving post-interventional distributions via discovered parent sets; and Flow is the most sensitive because the edges are the players of the game, so any graph error propagates across all downstream paths simultaneously.

Full numeric tables are in Appendix A; scatter plots in Figure 3; feature-level magnitude and sign heatmaps in Appendix A.5–A.6. Two patterns hold across both datasets. First, sign disagreement D_{base} is more severe than magnitude divergence ΔM_{base}: injecting a causal graph flips the direction of a larger share of attributions than it inflates their size, because sign inversions arise whenever a change crosses zero and many near-zero attributions sit close to that threshold. Second, the choice between PC and LiNGAM barely matters for the graph-versus-no-graph contrast: both discovered graphs produce nearly identical ΔM_{base} and D_{base} for a given Shapley method, so the dominant driver is the method's structural coupling to any graph, not the specific graph supplied.

ASV deviates the least from the Traditional baseline. On synthetic: D_{base} = 5.08–5.76%, ΔM_{base} = 0.68–0.86%. On Sachs: D_{base} = 16.80–17.00%, ΔM_{base} = 4.41–5.08%. The larger Sachs numbers reflect its compact 10-node network where each ordering constraint binds more feature pairs. **CSV** changes the sign of roughly a third of all attributions ($D_{\text{base}} \approx$ 33–35% on synthetic, $\approx$ 31–33% on Sachs). **Flow** shows the highest magnitude deviation (ΔM_{base} up to 14.17% of $\hat{\sigma}$ on Sachs) and the highest sign instability on synthetic (D_{base} = 36.58–37.68%).

4.3 Oracle Alignment

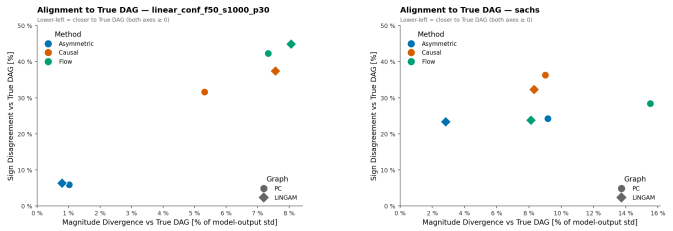
Oracle-alignment measures how closely each (method, discovered graph) recovers the attributions that would be obtained with the True DAG (tables: Appendix A; scatter: Figure 4; feature-level heatmaps: Appendix A.5–A.6). A consistent pattern emerges: the discovery algorithm with better F1 produces smaller oracle deviation on both dimensions. On synthetic, CSV under PC (F1 = 0.567) reaches D_{oracle} = 31.60%, ΔM_{oracle} = 5.32% vs. 37.46%, 7.57% under LiNGAM (F1 = 0.250). On Sachs the ranking reverses: ASV under LiNGAM (F1 = 0.326) achieves D_{oracle} = 23.40%, ΔM_{oracle} = 2.84% vs. 24.26%, 9.16% under PC (F1 = 0.167). Even imperfect graphs help when they are the better of the two available options.



(a) Synthetic. **(b)** Sachs.

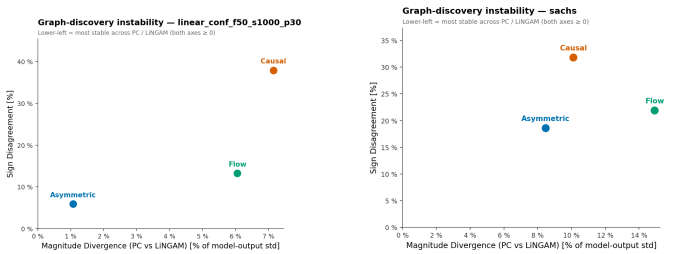
Figure 3: Deviation from Traditional Shapley. ASV clusters lower-left; CSV and Flow upper-right on both axes. Sachs spreads wider.

ASV achieves near-perfect oracle fidelity on synthetic ($\Delta M_{\text{oracle}} < 1.1\%$, $D_{\text{oracle}} < 6.3\%$) even under LiNGAM's severely degraded graph (F1 = 0.250, 81 false-positive edges): the 50-node sparse structure buffers ordering changes because spurious edges remove only orderings that were already rare in the 50! linear extension space. On Sachs this protection disappears, with only 10 nodes each mis-oriented edge eliminates a substantial fraction of valid orderings, so ASV+PC reaches ΔM_{oracle} = 9.16% of $\hat{\sigma}$, a level comparable to Shapley Flow on the same dataset. This focal divergence is driven by erk and pka; the feature-level heatmaps (Appendix A.5–A.6) confirm that oracle deviation is not uniform across features, and the neighbourhood errors of both nodes are examined in Section 4.5. **CSV and Flow** show substantial vulnerability on both tracks: on synthetic CSV+LiNGAM yields D_{oracle} = 37.46% and Flow reaches D_{oracle} = 42.26–44.91%, meaning close to half of all attribution signs are wrong relative to the True DAG. On Sachs the magnitude cost is even higher, Flow+PC reaches ΔM_{oracle} = 15.52% (the largest single oracle deviation in the experiment) and CSV exceeds 8% under both graphs, confirming that on a compact real-world network CSV and Flow in-



(a) Synthetic. **(b)** Sachs.

Figure 4: Deviation from the oracle DAG. ASV lower-left under both graphs; CSV and Flow upper-right. PC/LiNGAM markers spread wider on Sachs.



(a) Synthetic. **(b)** Sachs.

Figure 5: Cross-discovery instability (PC vs. LiNGAM). ASV lower-left (stable); CSV upper-right (unstable signs); Flow largest magnitude gap on Sachs.

herit the full cost of any discovery error rather than absorbing it.

4.4 Cross-Discovery Sensitivity

If a practitioner runs PC and LiNGAM on the same data and feeds each graph into the same Shapley method, how much do the explanations change? (Tables: Appendix A; scatter: Figure 5; feature-level heatmaps: Appendix A.5–A.6.) The answer depends heavily on the Shapley method. On Sachs, Flow's magnitude divergence between the two graphs (ΔM_{disc} = 14.92% of $\hat{\sigma}$) is nearly double ASV's (8.46%), meaning the choice of discovery algorithm alone makes Flow attributions twice as volatile. On the sign axis, switching from ASV to CSV on Sachs determines whether swapping graphs flips 18.60% or 31.80% of attribution signs. With these metrics, a practitioner can now form a quantitative expectation of how much their attributions might change between their two available graph inputs, and spot possible sources of divergence by inspecting the feature-level view, which can guide them to structural weaknesses in their discovered graph and inform a decision on whether that level of instability is acceptable for their use case.

ASV shows the lowest cross-discovery sensitivity on both tracks. On synthetic: ΔM_{disc} = 1.07%, D_{disc} = 5.93%. On Sachs sensitivity increases (ΔM_{disc} = 8.46%, D_{disc} = 18.60%) as the compact network makes different edge sets impose materially different topological constraints. **CSV** carries the largest sign instability (D_{disc} = 37.94% synthetic, 31.80% Sachs): interventional conditioning inverts attribution signs whenever parent sets differ between graphs. **Flow** carries the largest magnitude instability on Sachs (ΔM_{disc} = 14.92%).

4.5 Case Studies: When Graph Errors Matter Most

Three archetypal error mechanisms illustrate the micro-level behaviour observed across both experimental tracks. Each is identified by tracing a large global divergence, then inspecting the most prominent feature-level metric back and an instance-level view.

4.5.1 Out-Degree Inflation and Root Cause Overloading

Interventional (CSV) and edge-routing (Shapley Flow) methods over-inflate the attribution of any node that a discovered graph misrepresents as a high-degree root source. The clearest synthetic case is X24, a true direct parent of Y with 3 incoming and 4 outgoing edges in the True DAG. Both PC and LiNGAM strip its true parents and add spurious children (Figure 6), recasting it as an apparent high-degree root source; this inflates its outgoing-edge credit under Flow and its interventional parent set under CSV. X24's feature-level ΔM_{base} reaches 32.5% of $\hat{\sigma}$,on of the highest on the synthetic vs baseline track,and the SHAP scatter (Figure 7) confirms that both discovered-graph columns diverge substantially from the True-DAG oracle, also in a ΔM_{oracle} of around 32%, with inflated and partially sign-flipped values. Any feature producing a ΔM_{base} well above the dataset average deserves scrutiny of its discovered connections before any Structure-aware attribution is used downstream.

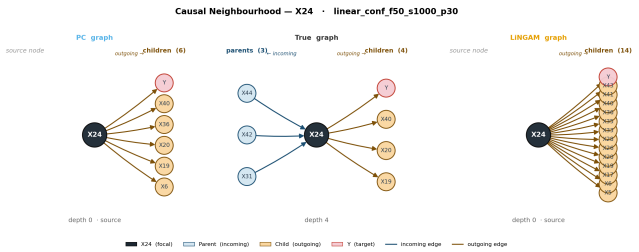


Figure 6: Causal neighbourhood of X24 (Synthetic). Both PC and LiNGAM strip true parents and add spurious children (6 and 14 out-edges respectively, 0 in-edges), converting a mid-graph node into an apparent root source. Blue = incoming, amber = outgoing.

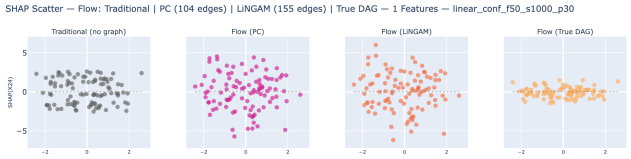


Figure 7: Shapley Flow SHAP scatter for X24 (Synthetic). Traditional shows a stable attribution pattern; both discovered-graph columns diverge substantially from the True-DAG oracle—the instance-level signature of out-degree inflation (ΔM_{base} = 32.5% of $\hat{\sigma}$).

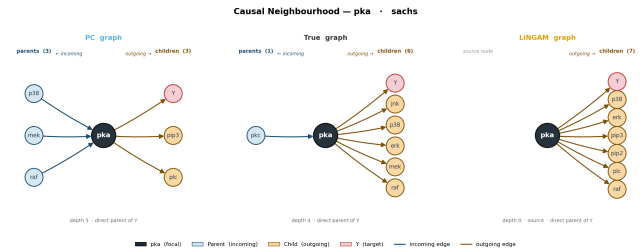


Figure 8: Causal neighbourhood of pka (Sachs). PC gives pka 3 incoming and 3 outgoing edges, reversing several true outgoing links. LiNGAM keeps it a near-root with 7 outgoing and 0 incoming edges, much closer to the consensus structure.

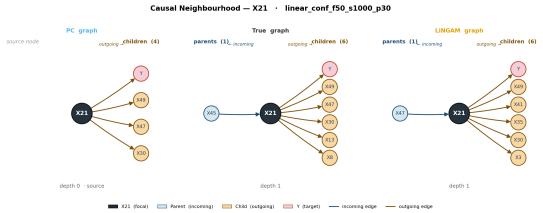


Figure 9: Causal neighbourhood of X21 (Synthetic). In the True DAG and under PC the edge runs $X_{21} \to X_{47}$. LiNGAM reverses it to $X_{47} \to X_{21}$, misdirecting attribution credit in both CSV and Flow.

The same mechanism appears on Sachs through protein pka, a near-root source (1 incoming, 6 outgoing edges in the consensus DAG). LiNGAM preserves this structure accurately, keeping pka as a near-pure root (ASV ΔM_{oracle} = 6.4% of $\hat{\sigma}$). PC reverses several of its outgoing edges into a 3-in/3-out configuration, and that mis-orientation is reflected in pka's ASV ΔM_{oracle} of 25.0% (Figure 8). A high ΔM_{base} (17.3% of $\hat{\sigma}$) combined with a large ΔM_{disc} (30.2% of $\hat{\sigma}$) forms the two-signal alarm: pka's discovered connections are both influential and contested across algorithms.

4.5.2 Directed Edge Inversion and Causal Credit Transfer

A single edge orientation error by one discovery algorithm can produce a sharply divergent attribution. In the True DAG the edge runs $X_{21} \to X_{47}$: X21 is a causal ancestor of X47, which is a dominant direct parent of Y. PC recovers this orientation correctly; LiNGAM reverses it to $X_{47} \to X_{21}$, placing X47 upstream of X21 (Figure 9).

Under PC, both CSV and Flow correctly treat X21 as an upstream node, assigning indirect credit for its causal influence through X47. Under LiNGAM the reversed edge injects X47 into X21's interventional parent set (CSV) and routes edge credit from X47 toward X21 (Flow), suppressing X47's attribution and inflating X21's. The SHAP scatters (Figure 10) confirm this: both CSV+PC and Flow+PC track the True-DAG oracle closely, while the LiNGAM variants diverge on both features. Feature-level ΔM_{disc} for CSV reaches 29.8% of $\hat{\sigma}$ on X47—the two-signal alarm indicating that a real edge's *orientation*, not its existence, must be resolved before using any structure-aware Shapley method.

4.5.3 Node Misspecification Captured by Experimental Metrics

Protein erk on Sachs is a good example where all three metric signals fire simultaneously, and where the framework's diagnostic value is clearest for a practitioner who has no ground-truth DAG. In the consensus DAG, erk is a downstream node receiving inputs from pka and mek. PC removes its true parent links entirely, leaving erk with outgoing edges only and recasting it as an isolated boundary source (Figure 11). Under Shapley Flow this isolation is

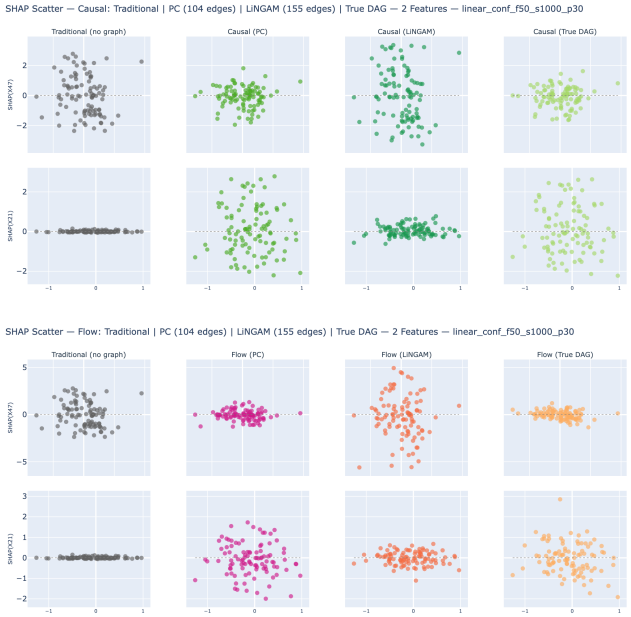


Figure 10: SHAP scatter for X21 and X47 (Synthetic). *Top* (Causal Shapley): under PC both features track the oracle; under LiNGAM, X47 is suppressed and X21 acquires inflated credit. *Bottom* (Shapley Flow): the same orientation-driven credit transfer appears; Flow+PC and Flow+True agree closely while Flow+LiNGAM diverges on both features.

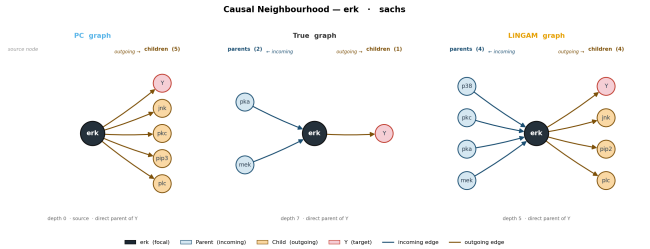


Figure 11: Causal neighbourhood of erk (Sachs). PC removes its true parent links (pka, mek) and isolates erk as a boundary source—driving Flow+PC's $\Delta M_{\text{oracle}} = 49.2\%$ of $\hat{\sigma}$.

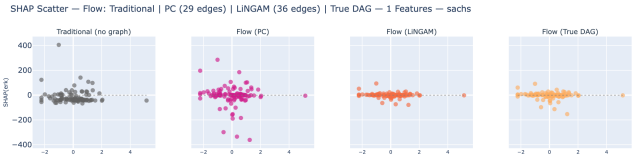


Figure 12: Shapley Flow SHAP scatter for erk (Sachs). All three graph-aware variants diverge from each other and from Traditional. Flow+PC inflates erk's attributions by treating it as an isolated source node; only Flow+True DAG tracks the correct downstream credit distribution.

catastrophic: with no incoming edges, erk accumulates all outgoing-edge credit without distributing any to upstream ancestors, yielding a feature-level $\Delta M_{\text{oracle}} = 49.2\%$ of $\hat{\sigma}$, the largest single oracle deviation in the entire experiment.

What makes erk especially important from a diagnostic standpoint is that both *observable* signals—those available without any ground-truth graph—also fire on this node. First, the feature-level ΔM_{base} shows elevated deviation from the Traditional Shapley baseline even for ASV, the most graph-robust method: erk appears among the highest-deviating features in the baseline magnitude heatmap (Appendix A.5), indicating that the injected graph is doing substantial work on erk's attribution, especially under the PC graph. Second, the feature-level ΔM_{disc} for erk is very large: the two discovery algorithms produce such different neighbourhoods that their attributions diverge substantially, with Flow reaching 57.5% of $\hat{\sigma}$ on erk in the cross-discovery heatmap (Appendix A.5). Crucially, even ASV, which stays near-zero for almost every other feature on Sachs, shows elevated ΔM_{disc} on erk (35.8% of $\hat{\sigma}$), driven by the large structural differences between what PC and LiNGAM discover for this protein. A practitioner running both algorithms and comparing feature-level attributions would immediately flag erk from the cross-discovery signal alone, with no oracle needed.

The SHAP scatter (Figure 12) makes the instance-level consequences visible: all three graph-aware variants produce materially different attribution patterns. Flow+PC inflates erk systematically by treating it as an isolated source. Flow+LiNGAM produces a distinct distortion: LiNGAM recovers some incoming structure for erk but the overall topology still diverges from the consensus, yielding attributions that match neither the PC column nor the True DAG oracle. The practical takeaway is direct: the combination of an elevated ΔM_{base} (graph injection is doing substantial work on erk) and an elevated ΔM_{disc} (the two discovered graphs fundamentally disagree on erk's position in the network) constitutes a two-signal alarm that requires structural validation before any structure-aware method is trusted for this node. The oracle confirms the damage, but the practitioner did not need it to identify the problem.

5 Conclusion

5.1 Summary of Findings

This work built an experimental pipeline to measure what happens to feature attributions when you plug an automatically discovered causal graph into a structure-aware Shapley method. The three methods fall into completely different sensitivity regimes. ASV is stable under graph errors: it only restricts the permutation space, leaving the observational value function untouched, so most orderings remain compatible across the true and discovered graphs and attributions barely move. CSV and Flow instead inject the graph directly into the conditioning set or the players themselves, so every false edge contaminates an attribution directly.

The two discovery algorithms swap rankings across datasets and neither reaches high accuracy on either. Even so, ΔM_{disc} flags the same contested nodes regardless of which algorithm performs better overall: on Sachs, ASV's cross-discovery sensitivity is the lowest of the three methods, yet its feature-level ΔM_{disc} still elevates on erk and pka, pointing a practitioner directly to the proteins whose discovered causal connections need further inspection. The key takeaway is that the practical decision is not about finding the best discovery algorithm: it is more productive to first check how sensitive the chosen Shapley method is to graph errors on the specific data at hand, and only invest in graph improvement where sensitivity is high.

The evaluation metrics reveal structure that is invisible when looking at attributions alone. ΔM_{base} flags nodes where the injected graph is doing substantial work, a signal that the structure-aware method has moved meaningfully away from the graph-free baseline on that feature. ΔM_{disc} flags nodes where the two discovery algorithms disagree on local structure, meaning the attribution is sensitive to which graph was chosen rather than to the model's predictive behaviour. When both signals are elevated on the same node, the graph structure for that feature is simultaneously influential and contested: the two-signal alarm that warrants structural validation before trusting any structure-aware method for that node.

5.2 Practical Recommendations

The most important contribution is the evaluation framework itself. Before this work, a practitioner adding a discovered graph to a Structure-aware Shapley computation had no systematic way to assess what that decision would cost in attribution quality. The six metrics at both dataset and feature level give a structured vocabulary for this assessment. The real world scenarios only the base and disc metrics are available; those are the ones the framework is really built around. The practical ordering is clear:

- **ASV is the recommended default.** It delivers oracle-level alignment with minimal deviation from the Traditional baseline, works well even with poor discovery results (Sachs PC: F1 = 0.167), and adds no computational cost above standard Monte Carlo SHAP. Regardless of which method is chosen, computing ΔM_{base} and ΔM_{disc} at the feature level is always worthwhile: together they identify which specific nodes have discovered connections that are both influential and contested, giving a targeted list of edges to validate or refine before committing to any structure-aware attribution.
- **CSV and Flow carry high risk and should only be used when the graph is specifically validated.** Acceptable conditions are: domain experts have reviewed and confirmed the contested causal connections identified by ΔM_{disc}, or the cross-discovery stability metrics show near-zero instability on the features that matter most. Even then the risk is substantial: CSV+LiNGAM on confounded synthetic produces $D_{\text{oracle}} = 37.5\%$ and $D_{\text{disc}} = 37.9\%$, meaning sign disagreements are not minor rescalings but full attribution inversions, and in a real deployment the oracle DAG is unknown, so there is no safety net to detect how far those inversions deviate from the true causal structure. An explanation that flips sign on a third of features tells the opposite causal story to the one the model's underlying system supports, making it difficult to defend under any form of audit.

5.3 Limitations and Future Work

The analysis covers a single synthetic functional form (linear confounded) and one real dataset; nonlinear and mixed functional forms, different graph densities, and larger sample sizes remain to be tested. All Shapley computations use $T = 100$ Monte Carlo samples. Future work should extend to nonlinear Shapley variants, explore ensemble approaches averaging attributions across multiple discovered graphs to reduce cross-discovery instability, and investigate whether active learning strategies could use the feature-level ΔM_{disc} signal to guide targeted graph refinement starting from the nodes the framework identifies as most contested.

References

[1] Samuel Baron. Explainable AI and causal understanding: Counterfactual approaches considered. *Minds and Machines*, 33:347–377, 2023. doi: 10.1007/s11023-023-09632-w.

[2] Christopher Frye, Colin Rowat, and Ilya Feige. Asymmetric Shapley values: Incorporating causal knowledge into model-agnostic explainability. In *Advances in Neural Information Processing Systems*, volume 34, pages 1229–1239, 2021.

[3] Clark Glymour, Kun Zhang, and Peter Spirtes. Review of causal discovery methods based on graphical models. *Frontiers in Genetics*, 10:524, 2019. doi: 10.3389/fgene.2019.00524.

[4] Tom Heskes, Evi Sijben, Ioan Gabriel Bucur, and Tom Claassen. Causal Shapley values: Exploiting causal knowledge to explain individual predictions of complex models. In *Advances in Neural Information Processing Systems*, volume 33, pages 4778–4789, 2020.

[5] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, pages 4765–4774, 2017.

[6] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

[7] Karen Sachs, Omar Perez, Dana Pe'er, Douglas A. Lauffenburger, and Garry P. Nolan. Causal protein-signaling networks derived from multiparameter single-cell data. *Science*, 308 (5721):523–529, 2005. doi: 10.1126/science.1105809.

[8] Lloyd S. Shapley. A value for n-person games. In Harold W. Kuhn and Albert W. Tucker, editors, *Contributions to the Theory of Games*, volume II, pages 307–317. Princeton University Press, 1953.

[9] Shohei Shimizu, Takanori Inazumi, Yasuhiro Sogourou, and Aapo Hyvärinen. DirectLiNGAM: A direct method for learning a linear non-Gaussian structural equation model. *Journal of Machine Learning Research*, 12:1225–1248,

2011.

[10] Peter Spirtes, Clark Glymour, and Richard Scheines. *Causation, Prediction, and Search*. MIT Press, 2nd edition, 2000.

[11] Jiaxuan Wang, Jenna Wiens, and Scott Lundberg. Shapley flow: A graph-based approach to interpreting model predictions. In *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 130 of *PMLR*, 2021.

A Detailed Results Tables

A.1 Baseline Deviation (vs. Traditional Shapley)

Table 3: Baseline Deviation – Linear-Conf Synthetic Dataset.

Method	Graph	D_{base}	ΔM_{base}
Asymmetric	PC	5.08%	0.68%
Asymmetric	LiNGAM	5.76%	0.86%
Causal	PC	35.30%	6.39%
Causal	LiNGAM	33.92%	5.84%
Flow	PC	36.58%	6.62%
Flow	LiNGAM	37.68%	6.61%

Table 4: Baseline Deviation – Sachs Cell Signaling Dataset.

Method	Graph	D_{base}	ΔM_{base}
Asymmetric	PC	16.80%	5.08%
Asymmetric	LiNGAM	17.00%	4.41%
Causal	PC	31.30%	7.83%
Causal	LiNGAM	33.10%	9.31%
Flow	PC	40.20%	12.68%
Flow	LiNGAM	41.50%	14.17%

A.2 Oracle Alignment (vs. True / Consensus DAG)

Table 5: Oracle Alignment – Linear-Conf Synthetic Dataset. D_{oracle} and ΔM_{oracle} in % of $\hat{\sigma}$.

Method	Graph	D_{oracle}	ΔM_{oracle}
Asymmetric	PC	5.88%	1.02%
Asymmetric	LiNGAM	6.30%	0.79%
Causal	PC	31.60%	5.32%
Causal	LiNGAM	37.46%	7.57%
Flow	PC	42.26%	7.34%
Flow	LiNGAM	44.91%	8.06%

Table 6: Oracle Alignment – Sachs Cell Signaling Dataset (oracle = consensus DAG). D_{oracle} and ΔM_{oracle} in % of $\hat{\sigma}$.

Method	Graph	D_{oracle}	ΔM_{oracle}
Asymmetric	PC	24.26%	9.16%
Asymmetric	LiNGAM	23.40%	2.84%
Causal	PC	36.30%	9.01%
Causal	LiNGAM	32.30%	8.31%
Flow	PC	28.36%	15.52%
Flow	LiNGAM	23.81%	8.12%

A.3 Cross-Discovery Sensitivity (PC vs. LiNGAM)

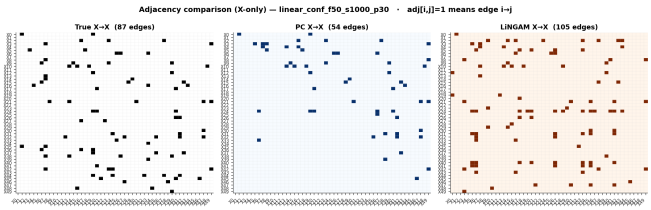
Table 7: PC vs. LiNGAM Sensitivity – Linear-Conf Synthetic Dataset.

Method	ΔM_{disc}	D_{disc}
Asymmetric	1.07%	5.93%
Causal	7.15%	37.94%
Flow	6.06%	13.27%

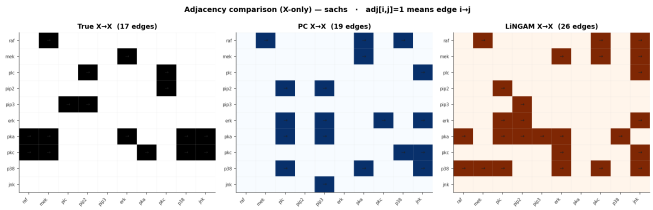
Table 8: PC vs. LiNGAM Sensitivity – Sachs Cell Signaling Dataset.

Method	ΔM_{disc}	D_{disc}
Asymmetric	8.46%	18.60%
Causal	10.11%	31.80%
Flow	14.92%	21.88%

A.4 Discovery Adjacency Matrices



(a) Synthetic: True DAG (87 edges), PC (54), LiNGAM (105). LiNGAM scatters spurious edges across the matrix.

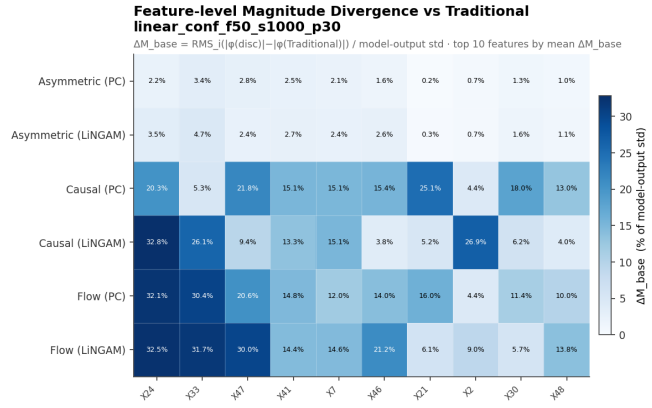


(b) Sachs: consensus DAG (17 edges), PC (19), LiNGAM (26). Both algorithms recover only a fraction of true links.

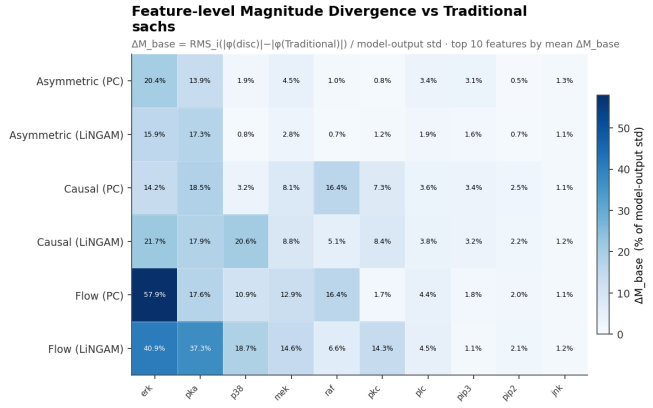
Figure 13: $X \to X$ **adjacency matrices.** $\text{adj}[i, j] = 1$ denotes $i \to j$. The discovery-quality ordering reverses between datasets.

A.5 Feature-Level Magnitude Heatmaps (ΔM)

Each heatmap shows ΔM_f^{ref} for the top 10 features by mean divergence. Rows are method-graph combinations; columns are features. Darker cells indicate larger attribution magnitude shifts. The same top features dominate all three comparisons.

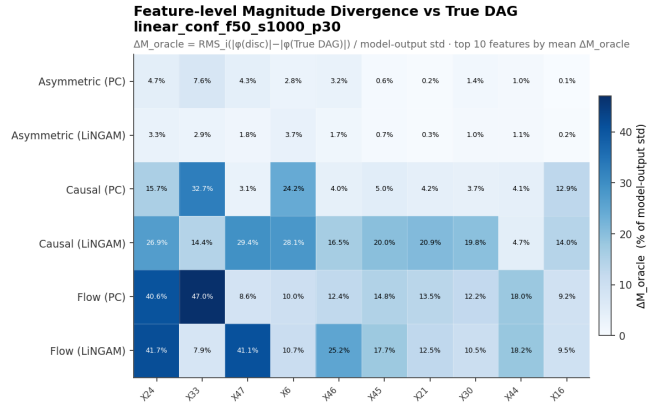


(a) Synthetic. Asymmetric rows near-zero throughout; Causal and Flow concentrate on X24, X33, X47.

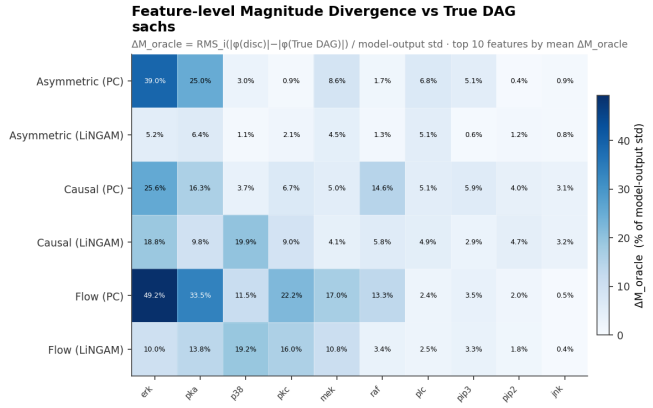


(b) Sachs. Deviation concentrates on erk and pka; Flow+PC produces the largest single shift (erk, 57.9% of $\hat{\sigma}$).

Figure 14: Feature-level ΔM_{base} vs. Traditional Shapley.

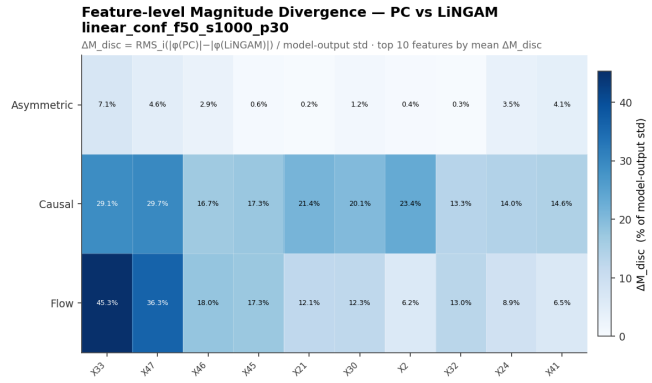


(a) Synthetic. Asymmetric rows near-zero; Causal and Flow concentrate oracle deviations on X24, X33, X47.

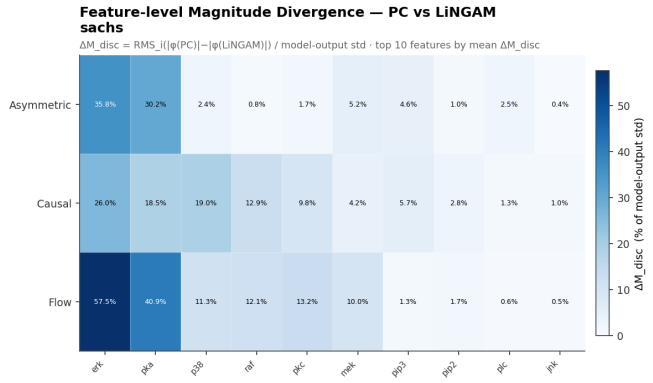


(b) Sachs. Flow+PC produces the largest single deviation on erk (49.2% of $\hat{\sigma}$).

Figure 15: Feature-level ΔM_{oracle} vs. the oracle DAG.



(a) Synthetic. Asymmetric near-zero; Causal and Flow concentrate on X33 and X47.

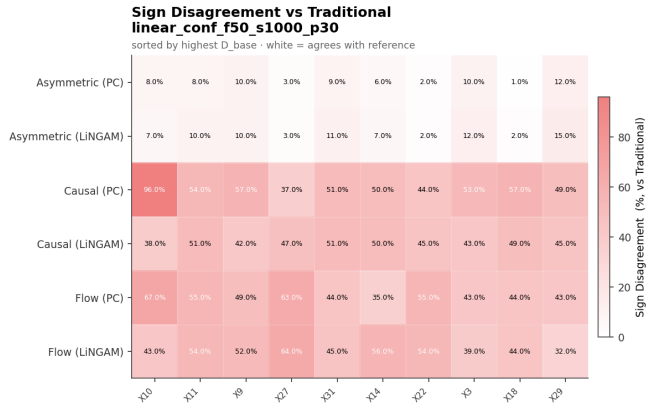


(b) Sachs. Sensitivity concentrates on erk and pka; Flow reaches 57.5% of $\hat{\sigma}$ on erk.

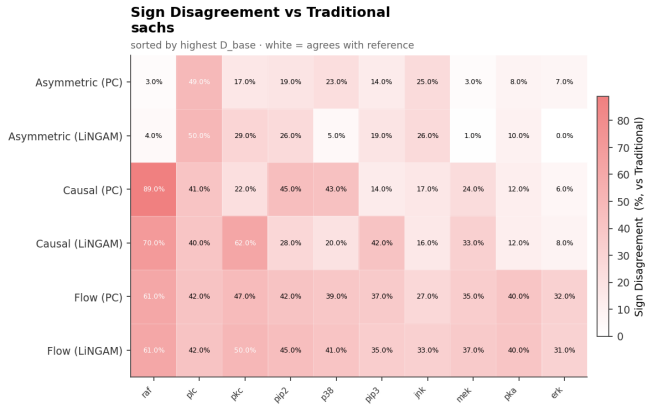
Figure 16: Feature-level ΔM_{disc} (PC vs. LiNGAM).

A.6 Feature-Level Sign Disagreement Heatmaps (D)

Each heatmap shows D_f^{ref} for the top 10 features by mean sign disagreement. Sign flips are qualitatively more damaging than magnitude shifts (they invert the story told to the end user), and they tend to spread across more features than magnitude changes because many near-zero attributions sit close to the sign boundary.

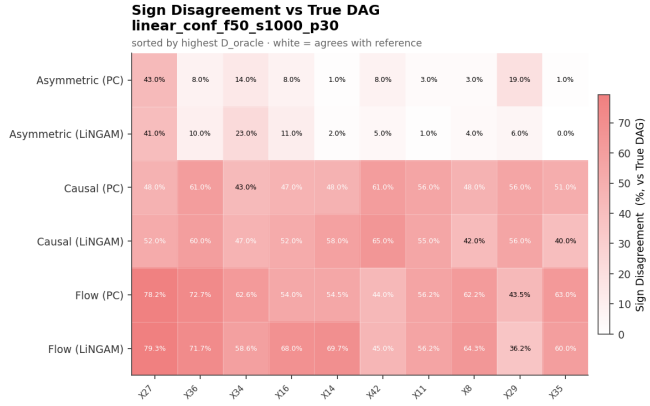


(a) Synthetic. Asymmetric rows near-zero; Causal and Flow concentrate sign flips on X24, X33, X47.

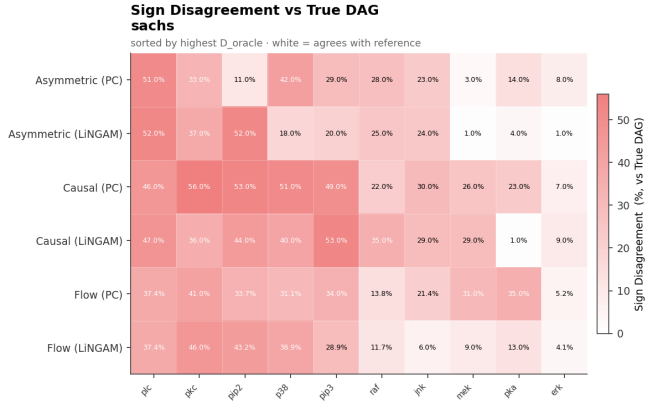


(b) Sachs. Sign disagreement concentrates on raf, plc, pkc; Causal+LiNGAM produces the highest per-feature flip rate vs. the baseline.

Figure 17: Feature-level D_{base} vs. Traditional Shapley.

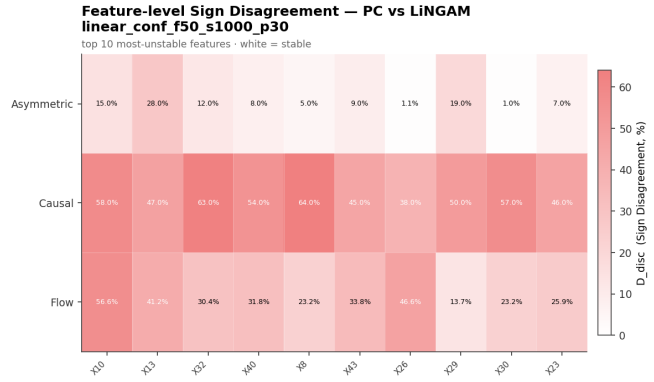


(a) Synthetic. Causal and Flow concentrate sign disagreement on X24, X33, X47 — the same features that carry the oracle magnitude deviations.

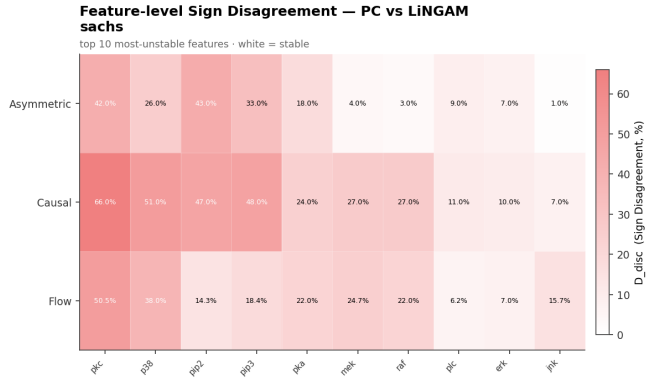


(b) Sachs. Sign disagreement vs. oracle concentrates on erk and pka; Flow+PC has the highest per-feature oracle sign-flip rate on erk.

Figure 18: Feature-level D_{oracle} vs. the oracle DAG.



(a) Synthetic. Causal shows the highest sign instability concentrated on X10 and X32; Flow's sign instability is moderate vs. its magnitude sensitivity.



(b) Sachs. Sign instability concentrates on pkc and p38; Causal Shapley shows the broadest sign disagreement across proteins.

Figure 19: Feature-level D_{disc} (PC vs. LiNGAM).

B Algorithm Pseudocode

The four main algorithms and five helper subroutines are listed below. They implement the methods described in Section 2.

B.1 Main Methods

Algorithm 1 Traditional Shapley (Monte Carlo)

INPUT: instance x, model f, background data D, permutations T
OUTPUT: attribution vector $\phi \in \mathbb{R}^n$

$\phi \leftarrow \mathbf{0}_n$; $v_0 \leftarrow \text{mean}(f(D))$
for $t = 1 \ldots T$ **do**
  $\pi \leftarrow \textsc{random\_permutation}(1 \ldots n)$
  $S \leftarrow \emptyset$; $v_{\text{prev}} \leftarrow v_0$
  for i **in** π **do**
    $S \leftarrow S \cup \{i\}$; $v_{\text{curr}} \leftarrow \textsc{coalition\_value}(x, S, f, D)$
    $\phi[i] \leftarrow \phi[i] + (v_{\text{curr}} - v_{\text{prev}})$; $v_{\text{prev}} \leftarrow v_{\text{curr}}$
  end for
end forreturn ϕ / T

Algorithm 2 Asymmetric Shapley

INPUT: instance x, model f, background D, feature DAG G, permutations T
OUTPUT: attribution vector $\phi \in \mathbb{R}^n$
PRECOMPUTE: `children[]`, `in_degree[]` from the X-only edges of G

$\phi \leftarrow \mathbf{0}_n$; $v_0 \leftarrow \text{mean}(f(D))$
for $t = 1 \dots T$ **do**
  $\pi \leftarrow$ SAMPLE_TOPOLOGICAL_ORDERING()
  $S \leftarrow \emptyset$; $v_{\text{prev}} \leftarrow v_0$
  for i **in** π **do**
    $S \leftarrow S \cup \{i\}$; $v_{\text{curr}} \leftarrow$ COALITION_VALUE(x, S, f, D)
    $\phi[i] \leftarrow \phi[i] + (v_{\text{curr}} - v_{\text{prev}})$; $v_{\text{prev}} \leftarrow v_{\text{curr}}$
  end for
end forreturn ϕ / T

Algorithm 3 Causal Shapley (post-interventional)

INPUT: instance x, model f, background D, adjacency A (X only), confounder list, outer permutations T, inner samples M
OUTPUT: attribution vector $\phi \in \mathbb{R}^n$
PRECOMPUTE: components, confounded[], parents[] $\leftarrow$ BUILD_COMPONENTS$(A$, confounder list$)$

$\mu \leftarrow \text{mean}(D)$; $\Sigma \leftarrow \text{cov}(D) + \varepsilon I$
$\phi \leftarrow \mathbf{0}_n$; $v_0 \leftarrow \text{mean}(f(D))$
for $t = 1 \dots T$ **do**
  $\pi \leftarrow$ SAMPLE_COMPONENT_TOPO_ORDERING(), expand to features
  $S \leftarrow \emptyset$; $v_{\text{prev}} \leftarrow v_0$
  for i **in** π **do**
    $S \leftarrow S \cup \{i\}$
    $v_{\text{curr}} \leftarrow \frac{1}{M} \sum_{m=1}^{M} f(\text{POST\_INTERVENTIONAL\_SAMPLE}(x, S, \dots))$
    $\phi[i] \leftarrow \phi[i] + (v_{\text{curr}} - v_{\text{prev}})$; $v_{\text{prev}} \leftarrow v_{\text{curr}}$
  end for
end forreturn ϕ / T

Algorithm 4 Shapley Flow (uniform edge-permutation)

INPUT: foreground instance x_{fg}, background row x_{bg}, graph adjacency ($n + 1$ nodes incl. Y), target index Y, trials T

OUTPUT: node attribution vector $\phi \in \mathbb{R}^n$ (Y excluded)

$E \leftarrow$ list of all directed edges; $\texttt{ec} \leftarrow \{e : 0.0 \mid e \in E\}$
for $t = 1 \ldots T$ **do**
  perm $\leftarrow$ RANDOM_PERMUTATION(E); active $\leftarrow []$
  $v_{\text{prev}} \leftarrow$ SYSTEM_VALUE($\emptyset, x_{\text{fg}}, x_{\text{bg}}$)
  for e **in** perm **do**
    active.append(e); $v_{\text{curr}} \leftarrow$ SYSTEM_VALUE(active, $x_{\text{fg}}, x_{\text{bg}}$)
    $\texttt{ec}[e] \leftarrow \texttt{ec}[e] + (v_{\text{curr}} - v_{\text{prev}})$; $v_{\text{prev}} \leftarrow v_{\text{curr}}$
  end for
end for
$\phi \leftarrow \mathbf{0}_n$
for each edge $(u \to v) \in E$ with $u \neq Y$ **do**
  $\phi[u] \leftarrow \phi[u] + \texttt{ec}[(u \to v)] \,/\, T$
end forreturn ϕ

B.2 Subroutines

Algorithm 5 COALITION_VALUE — $v(S) = \mathbb{E}[f(X) \mid X_S = x_S]$

function COALITION_VALUE(x, S, f, D)
  samples $\leftarrow$ copy(D)
  for each feature $j \in S$ **do**
    samples$[:, j] \leftarrow x[j]$
  end forreturn mean(f(samples))
end function

Algorithm 6 SAMPLE_TOPOLOGICAL_ORDERING — uniform random linear extension (Kahn)

PRECOMPUTE: (once): `children[]`, `in_degree[]` from the DAG (Y excluded).
If the feature DAG contains a cycle, disable constraints $\Rightarrow$ reduces to Traditional Shapley.

function SAMPLE_TOPOLOGICAL_ORDERING(`children`, `in_degree`)
  deg $\leftarrow$ copy(`in_degree`); ready $\leftarrow [i : \deg[i] = 0]$; order $\leftarrow []$
  while ready $\neq \emptyset$ **do**
    node $\leftarrow$ uniform pick from ready; swap-remove from ready
    order.append(node)
    for $c \in$ `children`[node] **do**
      $\deg[c]$ −= 1; **if** $\deg[c] = 0$: ready.append(c)
    end for
  end whilereturn order
end function

Algorithm 7 POST_INTERVENTIONAL_SAMPLE — draw from $P(X \mid \mathrm{do}(X_S = x_S))$

function POST_INTERVENTIONAL_SAMPLE(x, S, components, confounded[], parents[], μ, Σ)
    sample $\leftarrow \mathbf{0}_n$; **for** $j \in S$: sample$[j] \leftarrow x[j]$
    for component C in fixed topological order **do**
        missing $\leftarrow C \setminus S$; **if** missing $= \emptyset$: **continue**
        if confounded$[C]$ **then**
            for $j \in$ missing **do**
                sample$[j] \leftarrow$ GAUSSIAN_CONDITIONAL($\{j\}$, parents$[C]$, μ, Σ)
            end for
        else
            sample[missing] $\leftarrow$ GAUSSIAN_CONDITIONAL(missing, parents$[C] \cup (C \cap S)$, μ, Σ)
        end if
    end forreturn sample
end function

Algorithm 8 GAUSSIAN_CONDITIONAL — closed-form conditional Gaussian draw

function GAUSSIAN_CONDITIONAL(target A, cond B, vals b, μ, Σ)
    if $B = \emptyset$ **then return** draw from $\mathcal{N}(\mu_A, \Sigma_{AA})$
    end if
    $\Sigma_{BB}^{-} \leftarrow$ PSEUDO_INVERSE(Σ_{BB})
    $\mu_c \leftarrow \mu_A + \Sigma_{AB}\,\Sigma_{BB}^{-}\,(b - \mu_B)$
    $\Sigma_c \leftarrow \Sigma_{AA} - \Sigma_{AB}\,\Sigma_{BB}^{-}\,\Sigma_{BA}$; symmetrise; nudge eigenvalues $\geq \varepsilon$ **return** draw from $\mathcal{N}(\mu_c, \Sigma_c)$
end function

Algorithm 9 SYSTEM_VALUE — binary foreground/background activation (Shapley Flow)

function SYSTEM_VALUE(active_edges, x_{fg}, x_{bg})
    for each node i in the graph **do**
        if (i is a source **and** has an active outgoing edge) **or** (i has an active incoming edge) **or** (edge $i \to Y$ is active) **then**
            val$[i] \leftarrow x_{\mathrm{fg}}[i]$
        else
            val$[i] \leftarrow x_{\mathrm{bg}}[i]$
        end if
    end for
    drop Y from val **return** f(val)
end function